Contents

- Error Handling in Java Script
- Modules in Java Script
- Chaining Java Script Methods
- Promises

Error Handling in Java Script

Error Handling Functions

- JavaScript provides several mechanisms for error handling, including **try...catch blocks, throw** statement, and **finally** block.
- **try...catch Statement**:

a block of code to be tested for errors while it is being executed and a block of code to handle the error if it occurs.

```
try {
    // Code that may throw
an error
        } catch (error) {
    // Code to handle the
error
    }
```

Error Handling Functions

- **throw Statement**:

to create custom errors and throw them. This can be useful for signaling that an error has occurred in your code.

```
throw new Error 'Something went wrong'
```

Example

```
function divide(a, b) {
 if (b === 0) {
 throw new Error('Cannot divide by zero’);
 }
 return a / b;
}

try {
 let result = divide(10, 0);
   console.log(result);
} catch (error) {
   console.error('An error occurred:', error.message);
}
```

Error Handling Functions

- **finally Block**:

The finally block **follows a try...catch block** and contains code that will always be executed regardless of whether an error occurred or not.

```
try {
    console.log('Try block');
    throw new Error('Some error');
} catch (error) {
    console.error('Catch block:', error.message);
} finally {
    console.log('Finally block');
}
```

```
function sumOfFirstNOddNumbers(n) {
    if (typeof n !== 'number' || n <= 0 || !Number.isInteger(n)) {
        throw new Error('Invalid input: n must be a positive integer.');
    }
    let sum = 0;
    let count = 0;
    let currentNumber = 1;
    while (count < n) {
        if (currentNumber % 2 !== 0) {
            sum += currentNumber;
            count++;
        }
        currentNumber++;
    }
    return sum;
}
try {
    const n = 5; // Change this value to find sum of first n odd numbers
    const result = sumOfFirstNOddNumbers(n);
    console.log(`The sum of the first ${n} odd numbers is: ${result}`);
} catch (error) {
    console.error('Error:', error.message);
}
```

```
function processNumber(num) {
    if (typeof num !== 'number' || !Number.isInteger(num)) {
        throw new Error('Invalid input: The number must be an integer.');
    }

    // Perform some operation with the number
    console.log('Processing the number:', num);
}

try {
    const inputNumber = 3.14; // Change this value to test
    processNumber(inputNumber);
} catch (error) {
    console.error('Error:', error.message);
}
```

Modules in Java Script

Java Script Modules

- JavaScript modules allow you to **encapsulate related functionality, variables, and data, and then export and import** them as needed between different modules.
- There are two main types of modules in JavaScript:

1. **CommonJS Modules**
2. **ECMAScript Modules (ESM)**

CommonJS Modules

- CommonJS modules use **require() to import** modules and **module.exports to export** variables, functions, or objects.
- Example:

javascript

```javascript
// math.js
const add = (a, b) => a + b;
const subtract = (a, b) => a - b;

module.exports = {
    add,
    subtract
};
```

javascript

```javascript
// main.js
const { add, subtract } = require('./math');

console.log(add(5, 3)); // Output: 8
console.log(subtract(5, 3)); // Output: 2
```

ECMAScript Modules (ESM)

- ES modules use **import and export statements** for module loading and exporting.
- Example:

javascript

```javascript
// math.js
export const add = (a, b) => a + b;
export const subtract = (a, b) => a - b;
```

javascript

```javascript
// main.js
import { add, subtract } from './math.js';

console.log(add(5, 3)); // Output: 8
console.log(subtract(5, 3)); // Output: 2
```

Chaining Java Script Methods

Chaining JavaScript Methods

- a technique used **to invoke multiple methods on an object in a single statement,** where each method returns the modified object, allowing subsequent methods to be called on the result of the previous method.

Example

```javascript
const string = "   Hello, World!   ";

// Chaining methods to manipulate the string
const result = string
  .trim()           // Removes leading and trailing whitespace
  .toLowerCase()    // Converts the string to lowercase
  .replace(",", ""); // Replaces commas with an empty string

console.log(result); // Output: "hello world!"
```

- **trim()** removes the leading and trailing whitespace from the string.
- **toLowerCase()** converts the string to lowercase.
- **replace(",", "")** replaces any commas in the string with an empty string.
- Each method in the chain **operates on the result of the previous method.** This chaining allows for more concise and readable code.

javascript

```javascript
class Calculator {
  constructor(initialValue) {
    this.result = initialValue;
  }

  add(value) {
    this.result += value;
    return this; // Return the object itself for chaining
  }

  subtract(value) {
    this.result -= value;
    return this; // Return the object itself for chaining
  }

  multiply(value) {
    this.result *= value;
    return this; // Return the object itself for chaining
  }

  divide(value) {
    this.result /= value;
    return this; // Return the object itself for chaining
  }

  getResult() {
    return this.result;
  }
}

// Example usage of chaining methods
const calc = new Calculator(10);
const finalResult = calc.add(5).subtract(3).multiply(2).divide(4).getResult();
console.log(finalResult); // Output: 6.5
```

Write a JS Program for Chaining array methods to manipulate the array

1. Multiply each number by 2
2. Filter out numbers less than or equal to 5
3. Convert each number to a string
4. Join the numbers into a comma-separated string

```
const numbers = [1, 2, 3, 4, 5];
const result = numbers
  .map(x => x * 2)         // Multiply each number by 2
  .filter(x => x > 5)      // Filter out numbers less than or equal to 5
  .map(x => x.toString())  // Convert each number to a string
  .join(", ");             // Join the numbers into a comma-separated string

console.log(result); // Output: "6, 8, 10"
```

Promises

Promises

- a way **to handle asynchronous operations**, such as fetching data from a server, reading a file, or **executing a time-consuming task, without blocking the execution of the code.**
- Promises represent a value that may be available now, or in the future, or never.
- used when dealing with operations that take some time to complete and may succeed or fail.

3 States of Promises

1. **Pending**: Initial state, neither fulfilled nor rejected.
2. **Fulfilled**: The operation was completed successfully.
3. **Rejected**: The operation failed.

Syntax for creating a promise

```
const myPromise = new Promise((resolve, reject) =>
 {
// Asynchronous operation, e.g., fetching data from a server
// If successful, call resolve with the result
 // If an error occurs, call reject with the error
});
```

Example

- The promise **myPromise simulates an asynchronous operation** that resolves to a random number after 1 second.
- If the random number is greater than 0.5, the promise is fulfilled with the random number. Otherwise, it is rejected with an error.
- The **.then() method is called if the promise is fulfilled**, and the **.catch() method is called if the promise is rejected.**

```
const myPromise = new Promise((resolve, reject) => {
 setTimeout(() => {
  const randomNumber = Math.random();
  if (randomNumber > 0.5) {
   resolve(randomNumber);
  } else {
    reject(new Error("Random number is less than or equal to 0.5"));
  }
  }, 1000); // Simulate an asynchronous operation
});

// Handling promise
myPromise
  .then((result) => {
   console.log("Promise fulfilled with result:", result);
  })
  .catch((error) => {
   console.error("Promise rejected with error:", error);
  });
```

Exercise

1.Write a program using promises in JavaScript to load an image asynchronously.